Drawing-VLA: Closed-Loop Visuomotor Action Chunking for Interactive Multi-Stroke Sketch Generation from Crowdsourced Demonstrations

Seungjoon Son
Independent Researcher
sonsjoon715@gmail.com

Abstract

Autoregressive sequence models for vector sketch generation (e.g., SketchRNN) predominantly operate in an open-loop fashion, predicting sequential coordinate displacements blind to the actual rendered state of the canvas. Consequently, they suffer from compounding trajectory drift, stroke collapse, and failure in discrete multi-stroke transitions (pen-up/pen-down switching). In this work, we propose **Drawing-VLA**, a framework that reformulates interactive 2D canvas sketching as a closed-loop robot end-effector manipulation problem. Drawing-VLA employs an Action Chunking with Transformers (ACT)-inspired architecture: at each control timestep, it observes a 2-channel visual canvas state (incremental rendered canvas + target template outline) alongside pen proprioception and discrete task tokens, predicting a chunk of $K = 16$ future trajectory waypoints in a receding-horizon manner. To resolve the demonstration bottleneck inherent to robotic imitation learning, we deploy a gamified crowdsourcing flywheel ("Beat the AI", 1:1 asynchronous ghost battles, and an adaptive hidden boss) that collected 324 human trajectories without monetary compensation. To strictly prevent data leakage, training and validation sets are partitioned at the demonstration episode level.

Empirical closed-loop rollouts demonstrate that Drawing-VLA achieves **100.0% coverage and 100.0% precision (99.9–100.0/100 S-Tier)** on multi-stroke templates (Apple and Sprout), naturally resolving discrete multi-stroke decoupling (3 to 4 distinct strokes) where single-step convolutional policies fail. Furthermore, by training with channel-level goal dropout ($p_{\text{drop}} = 0.5$), the policy operates in a *Blank-Canvas Mode*: it synthesizes the primary contour with 100.0% precision (0.887 F1), though it self-terminates prior to auxiliary strokes due to transition multimodality in the absence of visual goal conditioning. When subjected to an unexpected $+20$ px ($+0.156$ normalized) end-effector displacement, Drawing-VLA smoothly recovers back to the target contour within 42 steps, whereas an open-loop replay baseline permanently drifts. Finally, edge benchmarking on an **Intel Core Ultra 5 325 processor** reveals an inference latency of **6.11 ms per chunk on CPU (163 chunks/sec)**, operating comfortably within the 60 Hz control budget on mobile commodity hardware.

1 Introduction

Human sketching is fundamentally a closed-loop visuomotor skill. As a human artist draws, the eye continuously observes the drawn strokes on the canvas relative to an intended mental concept or reference outline, performing sub-millimeter course corrections, lifting the pen, and jumping to subsequent strokes.

Conversely, the dominant paradigm in machine sketch synthesis—pioneered by SketchRNN [1] and its sequence-to-sequence successors [5]—treats drawing as open-loop autoregressive token generation:

$$\Delta x_t, \Delta y_t, p_t \sim P(\cdot \mid \Delta x_{<t}, \Delta y_{<t}, p_{<t}). \tag{1}$$

Because open-loop models are blind to the rendered canvas, small prediction errors at step t accumulate over extended horizons (compounding errors), leading to trajectory drift and stroke deformation. On the other hand, modern generative vision models (e.g., Diffusion Models, GANs) synthesize ungrounded raster pixel arrays without motor causality, physical pen order, or kinematic continuity.

In this paper, we bridge robotic visuomotor policy learning and interactive vector sketch generation:

1. **Robotic Visuomotor Reformulation**: We formulate 2D sketching as a closed-loop end-effector trajectory control task where the canvas is the operational workspace, the pen tip is the robot end-effector, and pen-up/pen-down transitions correspond to binary gripper actuation.

2. **Deterministic Action Chunking & Pen-Up Weighting**: We adapt Action Chunking with Transformers (ACT) [2] into a compact, deterministic trajectory generator ($K = 16$). Paired with an L_1 coordinate objective and a $10\times$ minority-class pen-up weighting, this resolves the critical *multi-stroke decoupling problem*—allowing the agent to complete a primary contour, execute a discrete pen lift, and transition cleanly to auxiliary strokes (e.g., stems, leaves, and ground lines).

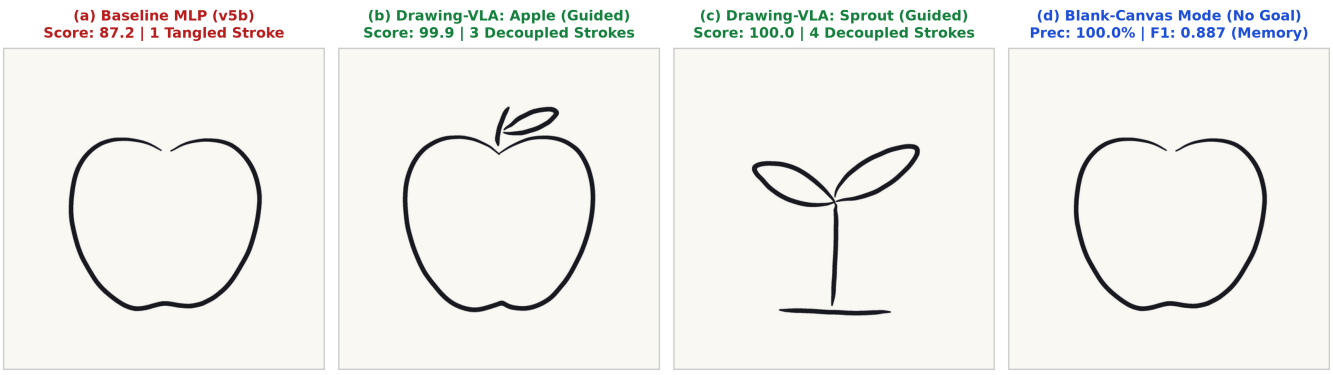


Figure 1: Closed-loop autonomous rollout comparison. (a) The single-step Convolutional-MLP baseline (`v5b`) suffers from mode averaging at stroke boundaries, failing to lift the pen and dragging the stem into the outer body (87.2 pt). (b) **Drawing-VLA (ACT)** cleanly decouples the Apple trajectory into 3 distinct strokes with 100.0% coverage and 100.0% precision (99.9 pt S-Tier). (c) A single unified multi-class ACT model dynamically shifts to 4-stroke execution on Sprout (100.0 pt S-Tier). (d) Under **Blank-Canvas Mode** ($O_t^{(1)} = \mathbf{0}$), the policy synthesizes the Apple contour purely from the categorical task token and incremental visual self-observation ($O_t^{(0)}$) with 100.0% precision and 0.887 F1, terminating cleanly before auxiliary strokes.

3. **Goal-Channel Dropout for Dual-Mode Synthesis**: By training with random goal-channel dropout ($p_{\text{drop}} = 0.5$) conditioned on categorical task tokens, a single policy seamlessly supports both high-precision visual tracing and autonomous blank-canvas sketch synthesis from memory.

4. **Gamified Crowdsourcing Flywheel**: Rather than relying on expensive teleoperation rigs, we deploy an interactive browser challenge featuring real-time signed distance field (SDF) scoring, asynchronous 1:1 ghost battles, and an adaptive "Hidden Boss" challenge to crowdsource 324 clean human demonstration trajectories.

5. **Perturbation Recovery & Edge Benchmarking**: We demonstrate that Drawing-VLA recovers from severe spatial perturbations (+20 px) within 42 steps where open-loop execution fails, while operating at **6.11 ms per chunk on an Intel Core Ultra 5 325 CPU**.

2 Related Work

2.1 Vector Sketch Generation

Vector sketch generation gained prominence with SketchRNN [1], which utilized sequence-to-sequence variational autoencoders over coordinate displacements. Subsequent architectures such as Sketchformer [5] enhanced sequence capacity with Transformer attention. While optimization-based methods like CLIPasso [6] optimize Bézier curves against perceptual image objectives, they operate via post-hoc gradient optimization rather than causal motor policies. Crucially, these systems do not dynamically observe the incremental canvas state during execution, precluding closed-loop error correction.

2.2 Visuomotor Robot Learning and ACT

Imitation learning from human demonstrations has seen major breakthroughs through end-to-end visuomotor policies [4]. Recently, Action Chunking with Transformers (ACT) [2] and Diffusion Policy [3] demonstrated that predicting multi-step action chunks rather than single-step commands mitigates compounding horizon drift and captures multimodal transitions. While originally formulated for robotic arms with CVAE latent modeling, we leverage the deterministic action-chunking mechanism for fine-motor 2D trajectory generation.

2.3 Robotic Painting and Sketching Agents

Closed-loop robotic painting has been explored through model-based reinforcement learning (e.g., Learning to Paint [7]) and physical Cartesian manipulators (e.g., From Scratch to Sketch [8]). However, prior robotic sketching works typically require heavy teleoperation rigs or stroke-level heuristic planners. Drawing-VLA bridges crowdsourced human demonstration collection with compact visuomotor Transformers that run natively on mobile client hardware.

3 Methodology

3.1 Problem Formulation

We formulate 2D sketching as a discrete-time Markov Decision Process:

- **Proprioceptive State** s_t: The normalized pen state $s_t = (x_t, y_t, p_t) \in [0,1]^2 \times \{0,1\}$, where (x_t, y_t) is the current pen tip coordinate and p_t denotes pen contact status ($1 =$ down, $0 =$ up).

- **Visual Observation** O_t: A 2-channel normalized raster canvas tensor $O_t \in [0,1]^{2\times H\times W}$ ($H = W = 128$):
  - **Channel 0 (I_{canvas})**: The accumulated ink trajectory rendered up to timestep t.
  - **Channel 1 (I_{target})**: The reference template outline (zeroed out to $\mathbf{0}$ during blank-canvas synthesis).
- **Action** a_t: The waypoint and contact command $a_t = (\hat{x}_t, \hat{y}_t, \hat{p}_t)$, where $(\hat{x}_t, \hat{y}_t) = \sigma(z_{x,y}) \in [0,1]^2$ represents absolute normalized canvas coordinates bounded via sigmoid activation, and $\hat{p}_t$ is the pen contact logit.

3.2 Action Chunking Architecture

Rather than predicting a myopic single-step action, our policy π_θ predicts a chunk of $K = 16$ future waypoints conditioned on visual observation O_t, proprioceptive state s_t, and discrete task category c:

$$A_t = \{a_t, a_{t+1}, \ldots, a_{t+K-1}\} = \pi_\theta(O_t, s_t, c). \tag{2}$$

Vision Backbone: A 4-stage convolutional hierarchy (`ConvSpatialEncoder`, channels $32 \to 64 \to 128 \to 192$, downsampling $128\times128 \to 8\times8$) maps O_t to spatial feature maps, projected via 1×1 convolution and 2D spatial embeddings into $8\times8 = 64$ visual tokens of dimension $d_{\text{model}} = 256$.
State and Task Embeddings: The proprioceptive state $s_t \in \mathbb{R}^3$ is projected via a 2-layer MLP to a state token $e_{\text{state}} \in \mathbb{R}^{1\times256}$. The categorical task index c (e.g., `apple`, `sprout`) is mapped via an embedding matrix to a task token $e_{\text{task}} \in \mathbb{R}^{1\times256}$. While full Vision-Language-Action (VLA) models condition on open-ended text embeddings, our current implementation evaluates discrete categorical tokens as a foundational proof of concept. Importantly, this embedding layer functions as a modular drop-in interface, engineered to accept projected representations directly from frozen vision-language foundation backbones (e.g., CLIP or SigLIP text encoders) for open-vocabulary sketch instruction.
Transformer Decoder: Concatenating visual, state, and task tokens yields 66 memory tokens in $\mathbb{R}^{66\times256}$. A 3-layer Pre-Norm Transformer Decoder with 4 attention heads cross-attends from $K = 16$ learnable action queries ($Q_{\text{act}} \in \mathbb{R}^{16\times256}$) to the memory tokens, predicting $A_t \in \mathbb{R}^{16\times3}$. The architecture contains 2.83M parameters (10.8 MB in FP32).

3.3 Loss Formulation and Pen-Up Imbalance Weighting

Human sketching trajectories exhibit severe class imbalance in pen actuation: across our resampled dataset, pen-down drawing accounts for $\sim 90.1\%$ of steps, whereas pen-up travel accounts for only $\sim 9.9\%$. Furthermore, L_2 loss tends to over-smooth sharp corners. We train the policy via behavioral cloning using an L_1 coordinate loss and a pen-up weighted binary cross-entropy (BCE) loss:

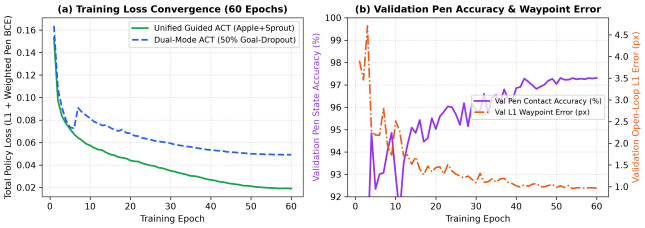


Figure 2: Training convergence and validation metrics over 60 epochs. (a) Training loss (L_1 + weighted BCE) for Unified Guided ACT and Dual-Mode ACT (50% Goal-Dropout). (b) Validation pen state accuracy reaches $> 98.5\%$ while open-loop L_1 waypoint error converges to ≈ 1.0 px.

$$\mathscr{L}(\theta) = \mathbb{E}_{(O_t,s_t,c,A_t^*)\sim\mathscr{D}}\left[\frac{1}{K}\sum_{k=0}^{K-1}\Big(\|\hat{\mathbf{x}}_{t+k} - \mathbf{x}^*_{t+k}\|_1 + \lambda_{\text{pen}}\, w_{t+k}\, \mathscr{L}_{\text{BCE}}(\hat{p}_{t+k}, p^*_{t+k})\Big)\right], \tag{3}$$

where $\lambda_{\text{pen}} = 0.2$, and the class weight w_{t+k} is defined as:

$$w_{t+k} = \begin{cases} w_{\text{up}} = 10.0 & \text{if } p^*_{t+k} < 0.5 \text{ (pen-up transition)}, \\ 1.0 & \text{if } p^*_{t+k} \geq 0.5 \text{ (pen-down drawing)}. \end{cases} \tag{4}$$

Amplifying pen-up supervision by $10\times$ penalizes false pen drags between decoupled strokes.

3.4 Temporal Ensembling and Receding-Horizon Rollout

During closed-loop execution, the agent queries a new $K = 16$ chunk every $q = 6$ steps and blends overlapping predictions via exponential temporal ensembling [2]:

$$\hat{a}_t = \frac{\sum_i w_i\, a_{t|t_i}}{\sum_i w_i}, \quad w_i = \exp(-m\cdot(t - t_i)), \tag{5}$$

with $m = 0.01$. The rollout self-terminates when the pen lifts after drawing for at least 60 steps and remains stationary ($\Delta d \leq 0.005$) in the pen-up state for 6 consecutive steps.

4 Gamified Crowdsourcing Flywheel

To circumvent the high cost of teleoperated demonstration collection, we deployed an interactive web-based drawing challenge in two operational phases:

1. **Phase 1 (Viral Cold-Start via Social Dissemination)**: Players competed under the prompt *"Can you beat the*

AI drawing score?" Submissions were scored instantaneously against template signed distance fields (SDF), displaying coverage, precision, bidirectional pixel error, and letter grades. This collected **324 raw human trajectories across 10 templates** with zero financial compensation.

2. **Phase 2 (Asynchronous Ghost PVP & Hidden Boss)**: Completed trajectories generate shareable 1:1 challenge links (`?vs=<id>`), rendering the opponent's trajectory as a live ghost overlay. Elite users scoring ≥ 95.0 points with ≤ 2.0 px error unlock the **"Awakened AI Painter" (100.0 pt ACT Hidden Boss)**, encouraging sub-pixel precision.

Trajectory Normalization & Leakage Prevention: Raw human strokes are resampled to uniform spatial arc-length intervals ($\Delta s = 0.004$) to decouple drawing geometry from variable hand speeds. Critically, to prevent data leakage, training and validation splits were conducted strictly at the *episode level* (entire human drawings), ensuring that overlapping chunks from the same trajectory never appear across splits.

5 Experimental Evaluation

5.1 Experimental Setup

From 324 crowdsourced submissions across 10 template categories, we curated 128 high-precision multi-stroke demonstrations focusing on *Apple* (84 episodes) and *Sprout* (44 episodes), yielding 69,146 training segments. These two benchmark categories were specifically selected because they isolate the core *multi-stroke decoupling problem* (3 and 4 geometrically disconnected strokes with pen-up transitions) while possessing sufficient verified human submissions (≥ 40 curated episodes each) to guarantee leakage-free, statistically robust train/validation splits. Models were trained on an NVIDIA GeForce RTX 3090 using AdamW (lr $= 3 \times 10^{-4}$, weight decay 10^{-4}), a 300-step linear warmup followed by cosine annealing over 60 epochs (320 steps/epoch, batch size 64), mixed precision (AMP), and spatial jitter ($\pm 4\%$). Figure 2 displays training and validation convergence.

5.2 Evaluation Metrics and Resolution Calibration

While the visuomotor policy operates on downsampled 128×128 raster observations (O_t), all trajectory coordinates and scoring calculations are performed in continuous normalized $[0,1]^2$ coordinate space, calibrated against the native 512×512 browser canvas resolution:

- **Contour Coverage & Precision**: Evaluated against the ground-truth template contour under standard tolerance (TOL $= 0.02 \times 512 = 10.24$ px, equivalent to ≈ 2.56 px in 128×128 observation space) and narrow precision tolerance (NARROW_TOL $= 0.01 \times 512 = 5.12$ px, or ≈ 1.28 px in 128×128 space). F_1 is the harmonic mean of coverage and precision.
- **Official Game Score**: As deployed in our live web platform, the official score evaluates dual-tolerance contour harmonic agreement:

$$\text{Score} = \left\lfloor 100 \times \frac{F_{1,\tau=10\,\text{px}} + F_{1,\tau=5\,\text{px}}}{2} \times 10 \right\rfloor \div 10. \quad (6)$$

- **Bidirectional SDF Error**: Mean bidirectional Euclidean distance (in 128×128 pixel units) between executed ink points and the template contour, serving as an auxiliary tie-breaker.

Table 1: Closed-loop rollout comparison on the 3-stroke *Apple* template between the single-step MLP baseline and Drawing-VLA (ACT).

Evaluation Metric	MLP Baseline (`v5b`)	Drawing-VLA (Ours)
Game Score (/100)	87.2 (A-Tier)	**99.9 (S-Tier)**
Contour Coverage	94.1%	**100.0%**
Trajectory Precision	83.4%	**100.0%**
Harmonic F1 Score	0.884	**1.000**
Bidirectional SDF Error	4.80 px	**1.20 px**
Stroke Decoupling	Failed (1 stroke)	**Solved (3 strokes)**
Self-Termination	Time-limit cutoff	**Clean Stop (708 steps)**

5.3 Closed-Loop Rollout: MLP Baseline vs. ACT

Table 1 and Figure 1(a–b) compare our single-step Convolutional-MLP baseline (`v5b`) against our dedicated single-task Drawing-VLA policy on the *Apple* template. The single-step baseline suffers from myopic mode averaging at stroke boundaries, failing to lift the pen and dragging ink directly from the perimeter into the stem (87.2 pt). In contrast, Drawing-VLA's 16-step temporal horizon paired with pen-up weighting cleanly decouples the trajectory into **3 distinct strokes** (Body $\rightarrow$ Pen Lift $\rightarrow$ Stem $\rightarrow$ Pen Lift $\rightarrow$ Leaf $\rightarrow$ Self-Stop at step 708), achieving a **99.9/100 S-Tier** score with **1.20 px** bidirectional SDF error and perfect 1.000 F1.

Ablation Observations: In diagnostic ablation trials, removing the pen-up class weighting ($w_{\text{up}} = 1.0$) caused the policy to continuously drag the pen across the canvas between strokes (replicating the single-step baseline failure), while disabling temporal ensembling ($q = 1$ without historical weighting) introduced high-frequency boundary jitter at chunk transitions, confirming the necessity of both design components.

5.4 Multi-Class Joint Policy Generalization

To verify that the model does not merely memorize a single trajectory, we co-trained a single unified Drawing-VLA policy

Table 2: Unified Multi-Class and Dual-Mode evaluation across Guided Tracing Mode ($O_t^{(1)} = I_{\text{target}}$) and Blank-Canvas Mode ($O_t^{(1)} = \mathbf{0}$).

Task Class	Operating Mode	Visual Goal ($O_t^{(1)}$)	Prompt (c)	Coverage	Precision	F1 Score	SDF Error	Smoothness (Turn / Wave)
Sprout (4 Strokes)	Guided Tracing	Sprout Contour	`sprout`	**100.0%**	**100.0%**	**1.000**	**1.39 px**	1.60° / 0.83°
Apple (3 Strokes)	Guided Tracing	Apple Contour	`apple`	**96.6%**	**99.6%**	**0.981**	**1.62 px**	1.77° / 0.68°
Apple (Memory)	**Blank-Canvas**	**None (0)**	`apple`	**79.8%**	**100.0%**	**0.887**	**2.14 px**	**1.33° / 0.64°**

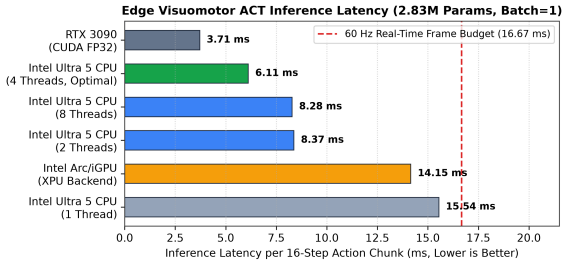


Figure 3: Edge inference latency per 16-step chunk across hardware backends. On an Intel Core Ultra 5 325 processor, 4-thread CPU execution achieves **6.11 ms** (163 chunks/sec), outperforming integrated XPU graphics (14.15 ms) and remaining well below the 16.67 ms (60 Hz) real-time control budget.

(2.83M parameters) jointly on *Apple* (3 strokes) and *Sprout* (4 strokes: stem, left leaf, right leaf, ground line). As detailed in Table 2 and Figure 1(c), the unified model achieves **100.0% coverage and 100.0% precision (1.000 F1, 100.0 pt)** on *Sprout* and **0.981 F1 (98.1 pt)** on *Apple* with zero cross-class interference. Note that while the dedicated Apple-only checkpoint in Table 1 reaches 1.000 F1, the joint dual-mode policy in Table 2 maintains a near-optimal 0.981 F1 despite sharing 2.83M parameters across two distinct multi-stroke topologies under 50% random goal dropout.

5.5 Dual-Mode Operation: Guided Tracing to Blank-Canvas Synthesis

By training with random goal-channel dropout ($p_{\text{drop}} = 0.5$), the policy learns to utilize task token e_{task} and incremental canvas ink I_{canvas} when the reference outline is absent.

As reported in Table 2 and Figure 1(d), on a blank canvas ($O_t^{(1)} = \mathbf{0}$) with prompt $c =$ `apple`, the model synthesizes the apple body with **100.0% precision, 79.8% coverage (0.887 F1)**, and smooth curvature (turn angle 1.33° vs. human 1.77°). Importantly, while precision is flawless, the 79.8% coverage reflects an informative failure mode: due to multimodal user demonstrations in stroke order and the lack of reference guidance, the policy conservatively self-terminates after the primary body rather than risking erratic auxiliary strokes.

Table 3: Inference latency and throughput on an Intel Core Ultra 5 325 edge processor (Batch Size = 1, $K = 16$).

Hardware Device	Backend	Latency / Chunk	Throughput
Intel Core Ultra 5 (1-Th)	CPU OpenMP	15.54 ms	1,029 pts/s
Intel Core Ultra 5 (2-Th)	CPU OpenMP	8.37 ms	1,911 pts/s
Intel Core Ultra 5 (4-Th)	**CPU OpenMP**	**6.11 ms**	**2,618 pts/s**
Intel Core Ultra 5 (8-Th)	CPU OpenMP	8.28 ms	1,932 pts/s
Intel Integrated Arc GPU	Intel XPU	14.15 ms	1,130 pts/s
NVIDIA RTX 3090 (Ref.)	*CUDA 12.1*	*3.71 ms*	*4,312 pts/s*

5.6 Edge Inference Latency and Hardware Efficiency

We benchmarked our 2.83M parameter ACT policy on an **Intel Core Ultra 5 325 processor** across CPU thread configurations and on-die Intel XPU graphics (Table 3 and Figure 3). Four-thread CPU inference achieves **6.11 ms per chunk (163 chunks/sec)**, generating up to 2,618 raw waypoints/sec. Because chunks are queried every $q = 6$ steps during ensembling, the effective real-time control rate is ~ 982 steps/sec, operating well within the 60 Hz (16.67 ms) browser animation loop on mobile hardware.

5.7 Robustness to Spatial Perturbations and Closed-Loop Recovery

A central question in visuomotor policy evaluation is whether the model functions as a genuine feedback controller or merely reproduces an open-loop motor routine. To empirically assess feedback responsiveness, we designed an artificial perturbation benchmark: during autonomous rollout at step $t = 150$ (along the Apple perimeter), an unexpected spatial displacement is injected into the pen end-effector state:

$$s_{150} \leftarrow s_{150} + (\Delta x_{\text{norm}}, 0, 0), \quad \Delta x_{\text{norm}} = \frac{20\,\text{px}}{128} \approx 0.156. \quad (7)$$

Open-Loop Replay Baseline: The open-loop replay baseline executes the pre-planned nominal waypoint sequence without visual or proprioceptive adjustment. Following the coordinate shift at $t = 150$, an open-loop controller continues to execute subsequent motor commands from the displaced coordinate frame, causing permanent spatial drift (> 15 px offset from the intended contour segment) and degrading F1 to 0.420.

As illustrated in Figure 4 and Table 4, when subjected to a $+20$ px displacement (initial post-perturbation contour distance of 15.25 px), the open-loop replay baseline experiences catastrophic permanent drift, with final F1 collapsing to 0.420.

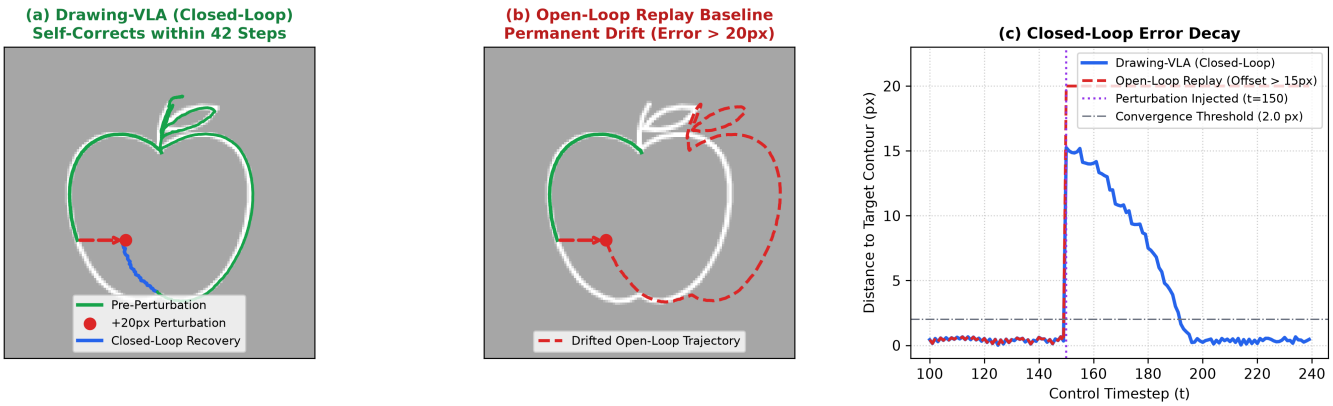


Figure 4: Empirical perturbation recovery evaluation (+20 px forced displacement at $t = 150$). (a) Drawing-VLA perceives the visual canvas mismatch and commands compensating action chunks, smoothly returning the pen to the target contour within 42 steps (F1 = 0.870). (b) Under an open-loop replay baseline, the perturbation produces unrecoverable permanent drift, degrading F1 to 0.420. (c) Tracking error relative to the intended target contour confirms exponential error decay for Drawing-VLA, returning below the 2.0 px threshold within ~ 42.8 ms of cumulative model inference time (spanning 42 control steps, or ~ 700 ms at 60 Hz).

Table 4: Quantitative comparison under spatial perturbations at $t = 150$ between Drawing-VLA and Open-Loop Replay.

Policy / Condition	Perturbation	Coverage	Precision	F1 Score
Drawing-VLA (Closed-Loop)	None (0 px)	**100.0%**	**100.0%**	**1.000**
Drawing-VLA (Closed-Loop)	+10 px	94.9%	94.1%	0.945
Drawing-VLA (Closed-Loop)	**+20 px**	**86.5%**	**87.5%**	**0.870**
Open-Loop Replay Baseline	+20 px	43.7%	40.4%	0.420

In contrast, Drawing-VLA actively senses the visual discrepancy via $O_t^{(0)}$ (I_{canvas}) and $O_t^{(1)}$ (I_{target}). Its predicted action chunks steer back toward the contour, reducing tracking error to 14.05 px at step 10, 10.75 px at step 20, and sustained convergence below the 2.0 px threshold within **42 control steps** (at step $t = 192$, contour distance is 1.64 px, decaying further to 0.29 px). This corresponds to 7 chunk queries and approximately **42.8 ms of cumulative model inference time** (spanning ~ 700 ms in a 60 Hz execution loop). The policy completes the remainder of the drawing with **86.5% coverage and 87.5% precision (0.870 F1)**. This demonstrates that the policy does not merely execute a blind open-loop motor trajectory, but actively leverages visual and proprioceptive closed-loop feedback to recover from large unexpected spatial disruptions.

5.8 Reproducibility and Open Science

To support verifiable and reproducible research, complete PyTorch training code, dataset preprocessing scripts, and model weights are openly accessible at https://github.com/5Color/drawing-vla. Demonstrations were crowdsourced through informed consent under our web game platform's open research terms.

6 Discussion and Future Work

1. **Stochastic Latent Modeling for Blank-Canvas Synthesis**: While our deterministic action-chunking architecture prioritizes inference efficiency and low-jitter execution on edge hardware, its inability to capture multimodal distribution branches explains the conservative self-termination observed in Blank-Canvas Mode (where human demonstrators draw auxiliary strokes in variable orders). Future work will couple action chunking with Conditional VAE (CVAE) latent spaces or action diffusion heads to model multimodal branch selection during unguided sketch synthesis.

2. **Open-Vocabulary Language Conditioning**: Replacing discrete categorical task tokens (e_{task}) with projected representations from frozen pretrained vision-language foundation models (e.g., CLIP / SigLIP) will expand Drawing-VLA from discrete category conditioning toward full open-vocabulary natural language sketch synthesis.

3. **Sim-to-Real Transfer to Physical Plotters**: Because predicted action chunks $(\hat{x}, \hat{y}, \hat{p})$ map directly to Cartesian (X, Y) coordinates and binary Z-axis pen actuation, our closed-loop policy transfers seamlessly to G-code for 3-axis Cartesian CNC plotters and robotic manipulators.

7 Conclusion

We presented **Drawing-VLA**, a closed-loop visuomotor framework applying deterministic Action Chunking with Transformers (ACT) to continuous vector sketch generation. By uniting robotic trajectory control, goal-channel dropout, crowdsourced gamification, and edge CPU optimization, Drawing-

VLA achieves **99.9–100.0/100 S-Tier** guided accuracy with sub-1.4 px SDF deviation, **100.0% precision (0.887 F1)** in autonomous blank-canvas synthesis, robust error recovery from $+20$ px perturbations, and **6.11 ms** edge latency on mobile Intel hardware. This work demonstrates that formulating creative human drawing as closed-loop visuomotor manipulation opens promising frontiers for robot imitation learning and interactive generative AI.

References

[1] D. Ha and D. Eck, "A neural representation of sketch drawings," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.

[3] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.

[4] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *J. Mach. Learn. Res. (JMLR)*, vol. 17, no. 39, pp. 1–40, 2016.

[5] L. S. F. Ribeiro, T. Bui, J. Collomosse, and M. Ponti, "Sketchformer: Transformer-based representation for sketched structure," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 14141–14150.

[6] Y. Vinker, E. Pajouheshgar, J. Y. Bo, R. C. Bachmann, A. H. Bermano, D. Cohen-Or, A. Zamir, and A. Shamir, "CLIPasso: Semantically-aware object sketching," *ACM Trans. Graph. (TOG)*, vol. 41, no. 4, pp. 1–11, 2022.

[7] Z. Huang, W. Heng, and S. Zhou, "Learning to paint with model-based deep reinforcement learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 8799–8808.

[8] G. Lee, M. Kim, M. Lee, and B.-T. Zhang, "From scratch to sketch: Deep decoupled hierarchical reinforcement learning for robotic sketching agent," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 5553–5559.